# Fire & Ice: Striking the Balance Between Efficiency and Privacy for Middleboxes over TLS

No Author Given

No Institute Given

**Abstract.** The widespread adoption of end-to-end encryption, like TLS, has diminished the effectiveness of traditional middleboxes (MBs). A common solution is the man-in-the-middle (MitM) approach, where MBs decrypt, inspect, and re-encrypt traffic, but this compromises the privacy of both endpoints, undermining TLS's primary goal. At USENIX'22, Grubbs et al. proposed ZKMBs, a solution allowing MBs to inspect traffic without decryption, preserving endpoint privacy. This method involves clients generating proofs to demonstrate compliance with security policies. Although verification is efficient, the overhead of session setup and proof generation makes ZKMBs impractical for widespread use.
In response, we introduce FIMBs, a new approach distinct from ZKMBs and MitM. FIMBs balance efficiency ("fire") and privacy ("ice") for MBs operating over TLS. Inspired by AEAD schemes like AES-GCM in TLS, FIMBs allow clients to selectively release keystreams for specific data blocks, enabling inspection while preserving the privacy of the remaining content. This method avoids ZKMBs' setup overhead, which would otherwise require a client to spend 21 seconds (and up to two hours for a two-million blocklist) during each new session. FIMBs introduce an overhead of approximately 3 seconds for a one-time setup and 300 milliseconds for keystream validation per block, making it a more promising solution from the client's perspective.

**Keywords:** Encrypted Traffic Inspection · Cryptography Proofs · Man-in-the-Middle · TLS Protocol.

## 1 Introduction

Many organizations endeavour to regulate their inbound and outbound network traffic to ensure security, compliance, and optimal performance. This regulation can be achieved by enforcing policies either directly on client devices or by deploying middleboxes (MBs) within the organization's network infrastructure. These MBs, such as firewalls, intrusion detection systems, and content filters, are widely recognized for their critical role in inspecting network traffic and blocking activities that violate established policies. Traditionally, these MBs have relied on the ability to scan network traffic in plaintext to detect and mitigate threats, enforce compliance, and optimize traffic flow. However, the increasing adoption of end-to-end encryption protocols, such as Transport Layer Security (TLS) such

as TLS 1.2 [17] and TLS 1.3 [60], has significantly diminished the effectiveness of traditional MBs. These encryption protocols ensure that data transmitted between endpoints is encrypted, thereby preventing MBs from viewing the content of the traffic, making it impossible to check for policy compliance or detect malicious activity.

To address this challenge, one of the most commonly implemented solutions is the man-in-the-middle (**MitM**) approach, e.g. SplitTLS [33]. In this configuration, MBs act as intermediary proxies between the client and the server. By intercepting and decrypting the encrypted traffic, MBs can inspect the data, enforce policies, and then re-encrypt the traffic before forwarding it to its intended destination. This approach allows MBs to perform their intended functions as if the traffic were not encrypted. However, the **MitM** approach introduces significant trade-offs. While it enables MBs to monitor and regulate all network traffic, it effectively compromises end-to-end encryption, leading to a complete loss of privacy for the clients. MBs gain access to all sensitive information transmitted over the network, raising serious privacy [47, 64] and security [14, 22, 57, 69] concerns. This intrusion can erode the trust between users and their service providers and may expose sensitive data to unauthorized access if the MBs are compromised.

Significant research has been devoted to addressing the challenge of balancing client privacy with the need for middlebox (MB) inspection. These efforts can be broadly categorized into two primary approaches. The first approach employs cryptographic tools to perform policy checks on encrypted data, as seen in various studies [24, 37, 39–42, 45, 52, 55, 63, 71, 75]. This method typically requires cooperation from the server and often involves modifications to standard TLS protocols. Although this approach preserves the integrity of encryption, it demands substantial alterations to well-established protocols, presenting significant challenges in implementation across diverse systems and infrastructures. In particular, modifying TLS is a complex and resource-intensive task, posing considerable barriers to widespread adoption.

Alternatively, a different approach involves designing MBs that operate within the existing TLS protocol framework by utilizing trusted hardware enclaves to perform policy checks [30, 25, 18, 20, 26, 31, 38, 58, 65, 70]. Trusted hardware enclaves, such as Intel's Software Guard Extensions (SGX), establish a secure environment within MBs, allowing encrypted data to be decrypted and inspected without exposing it to the broader system. While this approach enables MBs to function without modifying TLS, it introduces new security concerns. Despite their secure design, trusted hardware enclaves have been shown to possess a substantial attack surface, making them susceptible to various types of attacks [53]. Numerous studies have identified potential vulnerabilities and exploits that can compromise the integrity of these enclaves [27, 50, 61, 68, 66], thereby undermining the security and privacy they are intended to safeguard.

**Zero-Knowledge Techniques.** A novel approach, known as Zero Knowledge Middleboxes (**ZKMBs**), introduced by Grubbs *et al.* at USENIX'22 [28], represents a significant advancement in preserving client privacy while enabling MBs

inspection for specific applications. **ZKMBs** allow clients to prove to MBs that their encrypted traffic complies with policy requirements without disclosing the underlying plaintext. MBs verify these proofs, allowing only compliant traffic to proceed. Since the proof is zero-knowledge, MBs do not gain any information about the actual plaintext, aside from its compliance with policy guidelines. Importantly, **ZKMBs** do not require modifications to existing TLS 1.3 [60] or the use of trusted hardware, offering an elegant solution to the conflict between policy enforcement and privacy preservation. However, the initial implementation of **ZKMBs** demonstrated impractical performance, introducing substantial latency during the establishment of a new connection due to the requirement for proof of shared key derivation over TLS handshake. This significant delay renders **ZKMBs** unsuitable for practical use.

**Contribution and Challenges.** To evaluate the balance between efficiency and privacy in **ZKMBs** and **MitM**, we introduce **FIMBs** that offer a practical solution by adopting a selective inspection strategy, where specific data blocks are revealed for inspection while others remain private.

**FIMBs** aim to address the inefficiencies of **ZKMBs** by proposing a different protocol that can avoid such costly operations. Inspired by authenticated encryption with associated data (AEAD) schemes (§2.2) commonly used in standard TLS versions [17, 60]. Specifically, **FIMBs** utilize an *encrypt-then-authenticate* construction, such as AES-GCM with a minimum of 128-bit security, which is recommended by NIST as one of the preferred choices [8]. Given a message with a block size of 128-bit (16 bytes per block), **FIMBs** selectively release keystreams for decrypting specific TLS ciphertext block(s), allowing MBs to inspect block(s) in plaintext while preserving the privacy of the remaining content. This approach contrasts with **MitM**, which reveals everything. The process of generating keystreams and performing inspections is efficient and can be conducted in real-time. The keystream generation is integrated within the AEAD schemes when computing the TLS ciphertext tuple. MBs can obtain keystreams that decrypt certain blocks of the message into plaintext, making the inspection process straightforward.

The overhead introduced by **FIMBs** is primarily due to the validation of keystreams, which is necessary to address the identified security concerns (§3.3). In comparison, **ZKMBs** adopt a zero-knowledge proof technique to demonstrate the genuineness of key derivation (§3.3), which requires several seconds (and potentially hours if the policy includes a large blocklist) for setup and proof generation. At USENIX'24, Zhang *et al.* [77] improved **ZKMBs** by splitting the setup and proof generation into two phases. Although proof generation is now faster, the issue of setup time still persists. This highlights the trade-off between privacy and efficiency using **FIMBs** to omit such an expensive setup (§3.3).

We recognize the need for additional mechanisms to ensure the security of keystreams (§3.3), even though the method for generating them in standard TLS remains unchanged. Firstly, a dishonest client can easily send specially crafted keystreams to MBs. These keystreams may still allow MBs to decrypt message blocks, but the resulting data is neither legitimate nor authentic TLS ciphertext

tuple. This allows such attacks to evade inspection, effectively bypassing the intended function of the MBs.

One of the solutions is to adopt multi-party computation (MPC) to prove the generation of keystreams for some blocks of messages (§2.3). For example, we can use MPC to prove the obtained keystreams are correctly generated via an encryption function using the client's key and session nonce. It is notable that with MPC, only a one-time setup of approximately 3 seconds is required, unlike **ZKMBs** use zero-knowledge proofs that require several seconds of setup for each new connection.

While this approach may appear straightforward, it presents significant security challenges. Specifically, in TLS protocols that use AEAD schemes (*encrypt-then-authenticate* construction §2.2), e.g. AES-128-GCM [23] and ChaCha20/ Poly1305 [56] , it appears that semi-honest MBs, which adhere to the protocol while intentionally seeking sensitive data, could manipulate the session nonce by selecting a specific index for a chosen block to execute an MPC. Without rigorous verification, this could enable MBs to obtain some keystreams that reveal unauthorized blocks of the messages. Secondly, the session nonce is not inherently included in the TLS ciphertext tuple. Consequently, the client must transmit the session nonce to the MBs. This creates a vulnerability where a dishonest client could generate forged nonces, resulting in the creation of counterfeit keystreams. These forged keystreams could allow certain message blocks to bypass inspection, undermining the integrity of the process.

To mitigate the first attack by semi-honest MBs, we propose the adoption of boolean testing. This approach requires both the client and the MBs to provide their respective session nonces as inputs. The MPC will only proceed to generate the keystreams if and only if these input nonces are identical, ensuring that MBs cannot manipulate the session nonces to access unauthorized data.

To counter the second attack, we introduce an additional verification step that leverages the authentication tag from AEAD schemes (§2.2). In this process, the client is required to produce a proof tuple, which includes the keystreams for each block subject to inspection, the index list, the session nonce, and two additional keystreams specifically for authentication tag verification.

The core idea is to establish a binding relationship between the TLS ciphertext tuple and the proof tuple. By enforcing this relationship, we can effectively prevent a dishonest client from generating forged nonces or keystreams, thereby ensuring that all message blocks undergo proper inspection and that no content evades scrutiny. This approach enhances the security of the inspection process and demonstrates a significant advancement in balancing efficiency and privacy.

We demonstrate the practical applicability of **FIMBs** through use cases such as HTTP firewalling and DNS filtering, highlighting its efficiency and minimal overhead. Upon joining the network, the client retrieves system parameters from the MBs and completes a one-time setup process, which requires only approximately 3 seconds. In the context of HTTP firewalling, where the goal is to inspect the HTTP request version, we propose a policy that requires the inspection of 1–2 blocks of ciphertext, with each block comprising 16 bytes. **FIMBs**

introduce a minimal overhead of only 300–400 ms, in stark contrast to **ZKMBs**, which demand 21 seconds for setup when establishing a new session. In DNS filtering scenarios, **ZKMBs** can introduce significant delays, ranging from several seconds to as much as two hours if the blocklist includes two million domains. By comparison, **FIMBs** demonstrate remarkable efficiency, with an overhead of just 300–800 ms, depending on the size of the DNS query up to 86 bytes (6 blocks) in our experiments. These results underscore the significant contribution of **FIMBs** in offering a more efficient and practical solution for secure inspection while maintaining client privacy, making it a valuable advancement in this field.

### 1.1 Use Cases

In this work, our main objective is to enable MBs in enterprise or internet server providers (ISP) to inspect parts of the encrypted traffic while maintaining the privacy of the remaining content. This includes inspecting non-HTTPS traffic and applying blocklists to domains in encrypted DNS requests.

**Inspection on HTTPS Request.** In modern network environments, managing and securing outbound connections has become increasingly complex due to the widespread use of encryption. Traditional methods, such as port blocking, offer basic control by filtering traffic based on destination ports (e.g., 80 for HTTP and 443 for HTTPS) [62]. However, this approach is imprecise, often resulting in the overblocking of legitimate connections on non-standard ports or underblocking non-HTTP(S) traffic that uses standard ports.

To address these limitations, **FIMBs** aims to perform enhanced inspection of encrypted HTTP traffic. By leveraging techniques similar to **ZKMBs**, our **FIMBs** allow MBs to verify that outbound TLS connections indeed contain HTTP packets. This ensures that only legitimate HTTP traffic is permitted, even when it uses non-standard ports, without the need to inspect the entire content of the traffic, thus preserving privacy. This method strikes a balance between robust security enforcement and the need to avoid unnecessary blocking of valid connections.

**Encrypted DNS Request Filtering.** As encrypted DNS protocols like DNS-over-TLS (DoT) and DNS-over-HTTPS (DoH) become more prevalent, traditional methods of DNS filtering have struggled to keep pace. The encryption of DNS queries, while enhancing privacy, presents significant challenges for middleboxes that need to enforce security policies, such as blocking access to certain domains.

Building on the principles of traffic inspection used in the HTTP use case, **FIMBs** allow for the enforcement of DNS blocklists by identifying and filtering domain requests within encrypted traffic. This approach ensures that even in encrypted environments, where DNS queries are protected, the middlebox can enforce critical security measures without needing to inspect entire payloads or potentially other payloads that are not DNS queries. Furthermore, the blocklist can be kept hidden from the client, ensuring both security and privacy are maintained. By integrating this capability, the middlebox effectively manages encrypted DNS traffic, safeguarding the network while respecting user privacy.

# 2 Background

## 2.1 TLS Protocol

Transport Layer Security (TLS) is a family of protocols designed to ensure the end-to-end secure data exchanged between a client (such as a web browser) and a server (like a website). Many versions of TLS have been proposed over the years. According to the SSL Pulse project by the security company Qualys [59], as of May 2024, the adoption of TLS 1.3 [60] has continued to grow, with about 69.4% of popular websites supporting it, while TLS 1.2 [17] remains widely used, with nearly 99.9% support. It comprises a TLS handshake protocol, where the client and server negotiate and agree upon various cryptographic parameters, including the cipher suite to be used, which defines the key exchange, encryption, and hashing algorithms [60].

Once the cipher suite is selected, the handshake process continues with the generation of a shared secret, which is is derived through cryptographic operations agreed upon during the handshake and is not used directly for encryption. Instead, it is passed through a key derivation function to produce the session keys and nonces. These session keys and nonces are then used to encrypt and decrypt the application data exchanged between the client and server, ensuring confidentiality and integrity of the communication.

It is worth noting that **FIMBs** are independent of the TLS versions and the key exchange and hashing algorithms. Instead, **FIMBs** depend on the encryption algorithms employed, specifically the authenticated encryption with associated data (AEAD) schemes, which we will discuss in the following section (§2.2).

## 2.2 Authenticated Encryption with Associated Data

In this work, we focus on the widespread adoption of cipher suites that support authenticated encryption with associated data (AEAD) schemes with *encrypt-then-authenticate* construction, where data is first encrypted and then authenticated. Such encryption has become a cornerstone of modern TLS implementations, especially in TLS 1.2 [17] and TLS 1.3 [60]. The AEAD schemes consist of two algorithms as follows.

- $\texttt{Enc}_K(M, IV, AD) \rightarrow (C, T)$: An encryption algorithm that on input message $M$, nonce $IV$ (this is optional, most of the time $IV$ is randomly selected for security purpose), associated data $AD$, and a secret key $K$. It computes ciphertext $C$ and its authentication tag $T$. The resulting ciphertext tuple denotes as $(C, T)$.
- $\texttt{Dec}_K(C, IV, AD, T) \rightarrow M/\perp$: A decryption algorithm that on input $(C, IV, AD, T, K)$. It first checks the validity of $T$ to verify if $(C, IV, AD)$ has been corrupted or not. If it is invalid, the algorithm returns $\perp$. Otherwise, it proceeds to compute and return $M$.

*Correctness.* The correctness of an AEAD scheme holds if the decryption `Dec` is the inverse of the encryption `Enc`, such that $\texttt{Dec}_K(\texttt{Enc}_K(M, IV, AD), IV, AD) = M$.

**AES-128-GCM over TLS** We focus on AEAD schemes using *encrypt-then-authenticate* construction, such as AES-GCM [23] and ChaCha20 [56]. Specifically, we are adopting AES-128-GCM with 128-bit security, as it is not only recommended as a preferred choice by NIST [8] but also widely adopted in both TLS 1.2 [17] and TLS 1.3 [60] due to its support for hardware acceleration, such as AES-NI [3], which improves performance.

Suppose all messages are well-formatted, meaning messages transferred over the application layer are appended with *record type*, and the TLS handshake is completed, such that the session key and nonce pairs $(K, IV)$ are derived [60]. We briefly recap how the AES-128-GCM encryption algorithm [23] $\texttt{AES.Enc}_K(\boldsymbol{M}, IV, AD) \rightarrow (C, T)$ works as follows. Let $E.(\cdot) \rightarrow \{0,1\}^\lambda$ be an AES block cipher where $\lambda = 128$ is the security bits, $(K, IV)$ be a session key and nonce pair that derived from the TLS handshake. When a client wants to encrypt $n$-blocks of $\lambda$-bit block message $\boldsymbol{M} = (M_1, \cdots, M_n)$ with $M_j \in \{0,1\}^\lambda$ for $j \in [1, n]$. For all $i \in \{0, 1, \cdots, n\}$, let $IV_i = IV||i + 1$, we can compute keystreams $\boldsymbol{k} = (k_0, k_1, \cdots, k_n)$, such that

$$k_i = E_K(IV_i). \tag{1}$$

Then we compute the remaining ciphertext blocks $\boldsymbol{C} = (C_1, \cdots, C_n)$ and the authentication tag $T$ as follows. For all $j \in [1, n]$,

$$C_j = M_j \oplus k_j, \tag{2}$$
$$T = GHASH(AD, \boldsymbol{C}, H) \oplus k_0, \tag{3}$$

such that $H = E_K(0)$ where the input 0 is a zero padding to $\lambda$-bit block size and $GHASH(\cdot, \cdot, \cdot) \rightarrow \{0,1\}^\lambda$ is a function to produce a message authentication code based on multiplication in the Galois Field $\mathbb{F}_{2^{128}}$ [23]. The resulting ciphertext tuple over TLS is defined as $CT = (AD, \boldsymbol{C}, T)$ where $AD$ is publicly known associated data over TLS, which refers to the record header containing information like the record type, protocol version, and the record payload length. It is also worth noting that $IV$ is not sent over TLS.

As for the decryption $\texttt{AES.Dec}_K(\boldsymbol{C}, IV, AD, T) \rightarrow \boldsymbol{M} / \perp$, we need the knowledge of $(K, IV)$ to decrypt $CT = (AD, \boldsymbol{C}, T)$. In TLS, it is well understood that only parties with the shared secret may derive the session key and nonce $(K, IV)$. We first verify whether the authentication tag is valid or not by computing $k_0 = E_K(IV_0)$ where $IV_0 = IV||1$ and checking Equation (3). The algorithm returns $\perp$ if the above equation does not hold. Otherwise, we proceed to decrypt $\boldsymbol{C} = (C_1, \cdots, C_n)$, such that for all $j \in [1, n]$, we generate $k_j$ from Equation (1), and obtain $\boldsymbol{M} = (M_1, \cdots, M_n)$ from Equation (2), such that $M_j = C_j \oplus k_j$.

### 2.3 Multi-Party Computation

Secure multi-party computation (MPC) is a cryptographic technique that enables multiple parties to collaboratively compute a function while preserving the privacy of their inputs. Originating from Andrew Yao's pioneering works [73, 74],

MPC employs methods such as secret sharing [11, 12], homomorphic encryption [13], garbled circuits [9], and oblivious transfer [35, 36] to ensure both privacy and correctness.

For simplicity, we focus on a two-party scenario: the client **C** and the middlebox **MB** in **FIMBs**. At its core, MPC allows these two parties to jointly compute a function over their private inputs without revealing those inputs to each other. Each party contributes an input, and the protocol ensures that no more information is disclosed than what can be inferred from the output.

The protocol involves a computation $F(\cdot, \cdot)$, which takes inputs $X$ from party **C** and $Y$ from party **MB**, producing an output $Z$. The inputs $X$ and $Y$ remain private to parties **C** and **MB**, respectively. To securely compute $F$, both parties must agree on the function $F$. They utilize cryptographic techniques such as secret sharing, which divides a secret into multiple parts distributed among parties; homomorphic encryption, which allows computations on encrypted data; garbled circuits, where an encrypted function is evaluated without revealing inputs; oblivious transfer, ensuring one party obtains specific information without the sender knowing which; and zero-knowledge proofs, allowing verification of computations without disclosing inputs. These techniques ensure that the output $F(X, Y) \rightarrow Z$ accurately reflects the function of $F(\cdot, \cdot)$ without revealing the private inputs of the parties involved.

In this work, we adopt an advanced software framework, MP-SPDZ [34], designed to implement and experiment with various MPC protocols. MP-SPDZ supports many cryptographic protocols and is valuable for privacy-preserving computation. We do not use MPC for the whole AEAD schemes like AES-GCM, instead, we use it for the keystreams generation, i.e. AES 128-bit block encryption (16 bytes per block), as shown in Equation (1), i.e. $E_K(IV) \rightarrow k$ where $K$ is the session key and $IV$ is session nonce. In particular, **C** sets its input $X = K$, and **MB** sets its input $Y = IV$. At the end of the MPC, such that $F(X, Y) \rightarrow Z$, **MB** obtains a keystream $Z = k = E_K(IV)$.

## 3 Model and Overview

### 3.1 Threat Model

We assume that the underlying cryptographic primitives are secure and that adversaries have probabilistic polynomial-time computational power, in line with standard cryptographic practices. The setting for **FIMBs** is similar to that of **ZKMBs**, with a key difference in privacy protection. In **FIMBs**, we introduce a selective inspection mechanism that allows specific ciphertext block(s) to be revealed while keeping the other block(s) private. In contrast, **ZKMBs** ensure that all clients' content remains private, without any selective exposure.

Additionally, we assume that all entities in **FIMBs** follow the protocol correctly, and that the client and server do not collude to evade inspection, e.g. by sharing a pre-established key. **FIMBs** do not account for such attacks because it is impossible to prevent both end-to-end parties from communicating using their

pre-shared key. However, these kinds of encrypted channels can be blocked by enforcing adherence to the **FIMBs**. Therefore, it is reasonable to assume that the client may be a dishonest entity attempting to evade inspection.

On the other hand, we assume that the server is always an honest entity and is unaware of the existence of **FIMBs**. The TLS data between the client and server are encrypted using authentication encryption with associated data (AEAD) schemes using *encrypt-then-authenticate* construction (§2.2), specifically AES-128-GCM as stated in (§2.2), which is the recommended encryption algorithm in standard TLS nowadays [8].

Lastly, we assume that MBs are semi-honest entities that follow the protocol but intend to learn additional content beyond what it is authorized to access. In addition, we also assume that MBs and the server do not collude. Therefore, MBs will not modify the contents of TLS ciphertexts provided by the client.

### 3.2 System Flow

Our proposed **FIMBs** consist of the following four phases involving three entities: the client **C** who initiates the connection, the server **S** which serves as the receiving endpoint, and the middlebox **MB** which inspects the traffic. Figure 1 illustrates how **FIMBs** operate within the same framework as **ZKMBs** [28].

- **Setup**. Upon joining the network, **MB** forwards the necessary system parameters and policy to **C**. Such policy plays the role of the inspection and guides **C** to select message blocks to be revealed, i.e. given a sequence of $n$-blocks of the message, the policy identifies the indexes that require inspection
- **Establishment**. A standard TLS handshake protocol between **C** and **S**. The session keys and nonces $(K, IV)$ are derived for end-to-end encryption using the AEAD scheme with *encrypt-then-authenticate* construction (§2.2), e.g. the AES-128-GCM [23] or ChaCha20/Poly1305 [56].
- **Selection & Extraction**. Let $\boldsymbol{M} = (M_1, ..., M_n)$ be $n$-blocks of message such that $M_j \in \{0,1\}^{128}$ for $j \in [1, n]$. Then **C** computes ciphertext by running $\texttt{Enc}_K(\boldsymbol{M}, IV, AD) \rightarrow (\boldsymbol{C}, T)$. Based on the given policy and $\boldsymbol{M}$, **C** identifies the indexes that require inspection and stores them into a list $L$. For

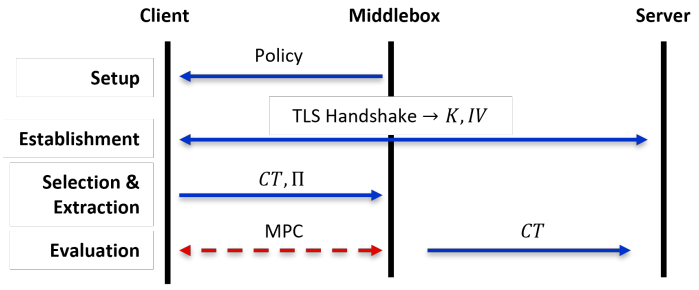


**Fig. 1.** Illustration of System Flow

every index $j \in L$, such that $j$-th block needs to be inspected, **C** generates keystreams $k_j$ – resulting a list of keystreams $\boldsymbol{k} = (k_{j_1}, \cdots, k_{j_L})$. Then **C** forwards a TLS ciphertext tuple $CT = (AD, \boldsymbol{C}, T)$ and a proof tuple $\Pi$, such that $\Pi$ consists of $(IV, \boldsymbol{k}, L)$ and additional proof elements (we further explain later in Section 3.3), to **MB**. Note that $IV$ is not sent in the standard TLS ciphertext tuple, but we bundle it as proof tuple $\Pi$.

– **Evaluation**. By receiving $CT = (AD, \boldsymbol{C}, T)$ and $\Pi$, **MB** is now able to inspect $j$-th block of messages. Once **MB** completes the inspection such that $j$-th block of messages obey the policy, both **C** and **MB** run MPC to claim $\Pi$ are genuinely generated and bind to $CT$. If the proof tuple $\Pi$ is valid, **MB** forwards $CT$ to the server. Otherwise, **MB** blocks the traffic and triggers an alert.

It is worth noting that the list of keystreams $\boldsymbol{k}$ can be intercepted by people who sniff in between **C** and **MB**. Although this is a drawback in **FIMBs**, our goal is to protect the remaining privacy of the contents other than those intended to be revealed. One of the trivial solutions that can overcome this issue is by establishing an additional secure channel between **C** and **MB** during the **Setup** so that it enables **C** to forward $\boldsymbol{k}$ securely.

### 3.3 Technical Overview

**Key Commitment Attacks** Most of the common encryption schemes in TLS are authentication encryption with associated data (AEAD) schemes, which do not support key commitment – a property that ensures a ciphertext can only be decrypted using the exact key originally used to encrypt it. This appears to be one of the main issues in generating proofs, which convinces MBs that the encrypted message obeys the policy. The adversary can produce a tuple of ciphertext, nonce, associated data, and authentication tag $(\boldsymbol{C}, IV, AD, T)$ that can be decrypted with at least two distinct key $K_1, K_2$ resulting in distinct messages $\boldsymbol{M}_1 \neq \boldsymbol{M}_2$, i.e. $\mathtt{Dec}_{K_1}(\boldsymbol{C}, IV, AD, T) = \boldsymbol{M}_1$ and $\mathtt{Dec}_{K_2}(\boldsymbol{C}, IV, AD, T) = \boldsymbol{M}_2$. We briefly recap some existing techniques that launch the key commitment attacks in high level [19, 29, 49, 16]. In short, the key commitment attacks can be generalized into two types, namely *Type I* a meaningful message if a nonce is chosen under the specific conditions, or *Type II* a random message on a fake key.

– *Type I.* The adversary chooses two messages $\boldsymbol{M}_1, \boldsymbol{M}_2$ and keys $K_1, K_2$. It then computes $(\boldsymbol{C}, IV, AD, T)$. This attack works under the conditions that (i) nonce $IV$ is correctly selected which preserves certain file structures, and (ii) the message can be restructured with random bytes in locations ignored by parsers for their respective file formats.
– *Type II.* The adversary chooses two distinct keys $K_1, K_2$ and a nonce $IV$, it then computes a ciphertext $\boldsymbol{C}$ on a chosen message $\boldsymbol{M}_1$ using $K_1$. This attack works under the condition that forged $(AD, T)$ can authenticate $\boldsymbol{C}$ under both keys and nonce $(K_1, K_2, IV)$ [19]. At the end, the ciphertext tuple

$(\boldsymbol{C}, IV, AD, T)$ can be decrypted using $K_1, K_2$ and obtain $\boldsymbol{M}_1, \boldsymbol{M}_2$, respectively, i.e. $\texttt{Enc}_{K_1}(\boldsymbol{M}_1, IV, AD) = \texttt{Enc}_{K_2}(\boldsymbol{M}_2, IV, AD) = (\boldsymbol{C}, T)$, such that $\boldsymbol{M}_2$ is some random messages.

Based on our observation, it is notable that *Type I*-key commitment attack is not an issue under the TLS encrypted channel because the nonce $IV$ is derived from the TLS handshake, so the attack condition does not meet. However, *Type II*-key commitment attack appears to be a challenging issue to be resolved if we are proving whether ciphertext contains messages that violate the policy, i.e. there is a match of message and policy. The ability to show a random message obeys the policy can pass the verification – violating the security of MBs' inspection.

**Our Alternative Way to Evade the Key Commitment Attacks** Fortunately, we identify that **FIMBs** are not susceptible to *Type II*-key commitment attack which enables a ciphertext to be decrypted as a random message under a different key. While **FIMBs** can inspect some message block(s) in plain, it is obvious that a random-look message block needs to trigger the alert. In particular, we introduce a new protocol that leverages the efficiency and privacy of **ZKMBs** and **MitM**.

The proposed **FIMBs** enable **MB** to inspect some block(s) of message in plain and keep the remaining blocks end-to-end encrypted. In particular, when **C** computes a ciphertext tuple with the TLS session key and nonce $(K, IV)$, e.g. using AES-128-GCM (16 bytes per block) such that $\texttt{AES.Enc}_K(\boldsymbol{M}, AD, IV, K) \rightarrow (\boldsymbol{C}, T)$ where $\boldsymbol{M} = (M_1, \cdots, M_n)$ is $n$-message block(s), $\boldsymbol{C} = (C_1, \cdots, C_n)$ is $n$-cipher block(s), and $AD$ is associated data, it will derive a list of keystreams $\boldsymbol{k}$ for some $j$-th block message $M_j$ in a list $L$ – allowing **MB** to inspect.

Technically, keystreams can be computed based on the definition in AES-128-GCM, i.e. Equation (1), such that for all $j \in \{1, \cdots, n\}$

$$E_K(IV_j) \rightarrow k_j, \text{ such that } IV_j = IV||j + 1.$$

It is notable that $C_j = M_j \oplus k_j$ and $E.(\cdot)$ is an AES block cipher. Besides, each $j$-th keystream $k_j$ is generated with distinct nonce $IV_j$. This said, although the same session key $K$ is used to derive $k_j$, it does not compromise the IND-CPA security of AES-128-GCM as every nonce $IV_j$ is distinct.

Let $n$ be the number of blocks for a message, and $L$ be a list of indexes. Given $n$-blocks of ciphertext $\boldsymbol{C} = (C_1, \cdots, C_n)$, and a list of keystreams $\boldsymbol{k} = (k_{j_1}, \cdots, k_{j_L})$, such that $j \in L$. **MB** can easily reveal $j$-th block(s) of message, such that $M_j = C_j \oplus k_j$.

**Remarkable Solutions to Identify Key Commitment Attacks: Channel Opening Sub-Circuit** It is notable that in the literature, **ZKMB** introduced the *channel opening sub-circuit*, denoted as $S_E$, to prove ciphertext is computed using the key derived from the TLS 1.3 handshake, so the key commitment attacks are identifiable. However, this causes the overhead for every session to

become very expensive – impractical as **C** needs to wait for 21-91 s to complete the setup, then only generates the proofs.

Recently, **Zombie** [77] was proposed in USENIX'24 to enhance the efficiency of **ZKMB**. They observed that the *channel opening sub-circuit* $S_E$ can be separated into two stages, such that $S_{E1}$: proof of derived keys and $S_{E2}$: proof of ciphertexts. In $S_{E1}$, which eventually is the setup phase but can be performed during idle time, **C** computes a proof that shows the re-derived keys with parameters from TLS handshake, then forwards the proof to **MB** (without knowing what messages to send).

In the later time, at stage $S_{E2}$, **C** proceeds to encrypt the messages and provide proof to **MB**, which claims the relations with proof at $S_{E1}$ and messages are policy compliance. The reported overheads at $S_{E2}$ to generate a proof is approximately 3.5× more efficient than **ZKMBs**, which is less than 400 ms. However, it is unclear how much overheads are still required at $S_{E1}$ to compute proof. Besides, this splitting mechanism does not seem to make the protocol more practical in the setting where **C** is browsing web services in scenarios where the session time out is short.

**Technical Challenges Encountered** Although our **FIMBs** omit the need for costly *channel opening sub-circuit*, we identify some new security issues that need to be resolved when **MB** obtains a TLS ciphertext tuple $CT$ and a proof tuple $\Pi$. In particular, we must ensure the following checks: (i) the semi-honest **MB** can verify the proof tuple $\Pi$ is genuinely computed from the ciphertext tuple $CT$ given by any **C** (which may be dishonest), and (ii) we need to ensure the semi-honest **MB** cannot reveal message blocks that are outside the policy. Specifically, if the $j$-th block(s) are to be revealed out of $n$-blocks under an index list $j \in L$, our **FIMBs** should prevent **MB** from observing any $j'$-th block that is not on the list, i.e. $j' \notin L$.

Recall that in **FIMBs**, the core basic idea for **C** to reveal $j$-th block of messages $M_j$ to **MB** is by releasing every of their keystreams $k_j$. By doing so, **MB** manage to compute $M_j = C_j \oplus k_j$ and perform the inspection with the defined policy, e.g. whether they are HTTP traffics, or the DNS queries fall under a blacklist. This mechanism looks trivial, however, it is not as it exists following attacks.

**Chosen keystreams attacks I.** Suppose a $j$-th block message $M_j = C_j \oplus E_K(IV||j+1)$ is violating the policy. We identify that a dishonest **C** can forge a keystream $k'$ reveals a resulting message that obeys the policy, such that $M' = C_j \oplus k'$ and $M' \neq M_j$.

**The Proposed Solution.** In **FIMBs**, we adopt an efficient MPC that can prove the derivation of keystreams as one of the solutions. In this case, **C** need to additionally forward the session $IV$ to **MB**. In particular, we are verifying the relationship of the keystream derivation using MPC, such that $E_K(IV_j) = k_j$ holds. This ensures that **C** is generating $k_j$ using the nonce for $j$-th block $IV_j = IV||j+1$ from its secret $K$.

If we consider **FIMBs** under the honest MBs setting, the above solution is good for an **MB** that honestly follows the protocol. However, we must consider MBs are at least semi-honest in the practice environment nowadays, such that the semi-honest **MB** will try to seek for keystreams that are out of $j$-th block in list $L$. Hence, we encounter the following attacks.

**Chosen IV attacks.** While verifying the keystreams using MPC to compute $E_K(IV_j) = k_j$, a semi-honest **MB** can select $j'$-th block that is out of list $L$ to seek for $E_K(IV_{j'}) = k_{j'}$, which revealing $j'$-th block of message $M_{j'} = C_{j'} \oplus k_{j'}$. We need additional mechanisms to prevent **MB** from selecting the $j'$-th index. Otherwise, it can reveal the plain message block that is not supposed to be inspected.

**The Proposed Solution.** We, therefore, require an additional step for MPC to have a boolean check that can verify the input $IV$ from both **C** and **MB** is equivalent. Then only they proceed to compute $k_j = E_K(IV_j)$, such that $IV_j = IV||j+1$. We now manage to prevent the semi-honest **MB** from revealing $j'$-th block message that is out of the list $L$.

**Chosen keystreams attacks II.** There is still an issue such that a dishonest **C** actually forwards a well-crafted nonce $IV'$ that is not from the TLS session nonce, such that the resulting keystream decrypts ciphertext that obeys the policy. A dishonest **C** can easily compute $IV'$ by running the inverse AES block cipher such that $D_K(k') \rightarrow IV'$, where $k' = M' \oplus C_j$ is crafted as in **Chosen keystreams attacks I**. During the evaluation, upon receiving $k', IV'$ from part of the proofs material $\Pi$, **MB** perform the MPC based on the above mechanisms like checking the input $IV'$ and the resulting $k' = E_K(IV')$ evades the policy as $M' = C_j \oplus k'$.

We note that this attack works when there is only a single block of messages that needs to be inspected. In the cases where multiple blocks require inspection, the collusion of nonces that match the sequence is negligible. For example, let $IV'$ be a well-craft nonce. Suppose we are inspecting $j$-th and $(j+1)$-th block, that means **C** needs to release $k'_j = E_K(IV'_j)$ and $k'_{j+1} = E_K(IV'_{j+1})$ under the same $IV'$. A dishonest **C** can easily compute $k'_j = M'_j \oplus C_j$, such that $M'_j$ obeys the policy, however, it is likely that $k'_{j+1} = M'_{j+1} \oplus C_{j+1}$ may result a random message block $M'_{j+1}$ that looks suspicious from the view of **MB**. Although this can be easily resolved by enforcing **MB** to detect if messages look suspicious, for completeness purposes, we still need a mechanism to prevent such attacks.

**The Proposed Solution.** This is where we are releasing additional proof elements $(H, k_0)$, such that $E_K(0) = H$ and $E_K(IV_0) = k_0$ where $IV_0 = IV||1$, to verify the bind relationship of $(\boldsymbol{k}, IV, T)$.

In the complete **FIMBs** protocol, we perform the following three proofs. They are (i) to prove if both parties are using the same $IV$; (ii) to prove if $H = E_K(0)$ and $k_i = E_K(IV_i)$ for $i \in [1, \cdots, n]$; and (iii) to prove if $T = GHASH(AD, \boldsymbol{C}, E_K(0)) \oplus E_K(IV_0)$. Recall that $(AD, T)$ is an associated data and authentication tag tuple from the TLS tuple.

It is notable that (i) and (ii) were already discussed in the aforementioned solutions. We now remain to discuss how (iii) works. Given a TLS ciphertext tuple $CT = (AD, \boldsymbol{C}, T)$ and proof tuple $\Pi = (IV, \boldsymbol{k}, L, H, k_0)$, this enables **MB** to check if the Equation (3) holds $T = GHASH(AD, \boldsymbol{C}, H) \oplus k_0$.

## 4 The Proposed FIMBs

We consider a similar setting as **ZKMBs** [28], discussed in (§3.2), which involves three entities: the client (**C**), the server (**S**), and the middlebox (**MB**).

In **FIMBs**, the main goal of **C** is to establish communication with **S** via a secure channel like TLS 1.2 [17] or TLS 1.3 [60] that supports AEAD scheme with *encrypt-then-authenticate* construction, such as AES-128-GCM §2.2. Conversely, **MB** aims to enforce a specific policy, commonly involving the dropping of traffic that deviates from this policy, i.e. HTTP firewalling and DNS filtering. Such policy enforcement typically dictates what kind of services and requests that **C** can make. It is notable that **S** is unaware that **FIMBs** protocol is in use.

While **MitM** approach inspects all traffic efficiently but with no privacy due to its ability to decrypt everything, in **ZKMBs**, **MB** inspects in a privacy-preserving manner but it is inefficient due to long setup time. We attempt to strike a balance between efficiency and privacy. In particular, the proposed **FIMBs** protocol enables MB to inspect certain blocks of messages only on the condition that **C** reveals designated blocks based on policy.

In general, the basic flow of **FIMBs** begins with **C** downloading a network policy specification from **MB** upon joining the network. This specification defines a policy-relevant scope in which **C** must reveal certain keystreams and proofs to **MB** for the application of HTTP firewalling and DNS filtering, for each encrypted channel to a remote **S**. With the knowledge of keystreams, **MB** can now perform inspections for the block(s) of TLS ciphertexts. Then, for each proof, **C** and **MB** perform an evaluation protocol to verify the binding relationship between TLS ciphertexts and keystreams. The following subsections define the four stages in **FIMBs** protocol.

### 4.1 Setup

When a client **C** joins a network, there exists a middlebox **MB** forwards the system parameters and the defined policy to **C**. This policy may include, for example, a browser extension designed to perform header manipulation. Given a sequence of $n$ message blocks, the policy identifies the indexes that require inspection. We further discuss this in §5 for applications like HTTPS firewalling and encrypted DNS filtering.

### 4.2 Establishment

In this stage, **C** establishes a secure end-to-end connection with **S** by running the TLS handshake protocol to negotiate the cipher suites required for key exchange, encryption, and hashing algorithms.

We specify that **C** must enforce the selection of cipher suites that are AEAD schemes with *encrypt-then-authenticate* construction, such as AES-128-GCM or ChaCha20/Poly1305. For the current implementation of **FIMBs**, we assume the selection must be AES-128-GCM under TLS 1.2 or TLS 1.3. At the end of this stage, both **C** and **S** obtain a shared secret, which can be used to derive the session key and nonce pair $(K, IV)$ for AES-128-GCM.

### 4.3 Selection & Extraction

Given the input of $n$ message blocks $\boldsymbol{M} = (M_1, \cdots, M_n)$ such that each block has a size of 128-bit (16 bytes), **C** executes AES-128-GCM to obtain a ciphertext and tag tuple which denotes as $\texttt{AES.Enc}_K(\boldsymbol{M}, IV, AD) \rightarrow (\boldsymbol{C}, T)$ (defined §2.2). The TLS ciphertext tuple is then set as $CT = (AD, \boldsymbol{C}, T)$. It is worth noting that in practice, the exact value of $AD$ reflects the header of the network payload that is sent via the TLS traffic, e.g. it indicates the data type, the legacy protocol version, and the length of the data (size of $\boldsymbol{C}$ and $T$).

A list of indexes $L$ is chosen based on the message and given policy in the setup phase. We further explain how the indexes are selected in §5. Suppose $j \in L$. Let **C** computes keystream $k_j$ for $j$-th block of $M_j$, such that

$$E_k(IV_j) \rightarrow k_j.$$

Recall that in AES-128-GCM, the $j$-th block of messages is encrypted with $IV_j = IV||j+1$, such that $C_j = M_j \oplus E_K(IV_j)$. In addition, **C** also computes $H = E_K(0)$ and $k_0 = E_K(IV_1)$ for authentication. At the end, **C** releases TLS ciphertext tuple $CT = (AD, \boldsymbol{C}, T)$ and proof tuple $\Pi = (IV, \boldsymbol{k}, L, H, k_0)$ to **MB**.

### 4.4 Evaluation

Given the TLS ciphertext tuple $CT = (AD, \boldsymbol{C}, T)$ and the proof tuple $\Pi = (IV, \boldsymbol{k}, L, H, k_0)$, **MB** now inspects the revealed message blocks by computing $M_j = C_j \oplus k_j$, such that $j \in L$, and checking with the policy (§5). **MB** blocks the traffic if it is being identified as violating the policy. Otherwise, **C** and **MB** execute the MPC to check the validity of $(CT, \Pi)$.

We perform the following checks and abort whenever any of them do not hold. Let $\boldsymbol{t} = (IV_0, IV_{j_1}, \cdots, IV_{j_L}, 0)$ be a list of session nonce with indexes. For every element in $\boldsymbol{t}$,

1. Boolean Check $X = Y$: The input $X$ from **C** and $Y$ from **MB** is equivalent, such that $X = Y$ for every item in $\boldsymbol{t}$.
2. AES Block Cipher $E_X(Y) \rightarrow Z$: The input $X = K$ is the session key from **C** and $Y$ from **MB** for every item in $\boldsymbol{t}$. It checks whether $E_K(IV_0) = k_0,\ E_K(IV_{j_1}) = k_{j_1}, \cdots,\ E_K(IV_{j_L}) = k_{j_L},\ E_K(0) = H$ hold.

Lastly, **MB** checks if $T = GHASH(AD, \boldsymbol{C}, H) \oplus k_0$ holds. If the equation holds, **MB** forwards $CT$ to **S**. Otherwise, **MB** drops the traffic and triggers the alert.

## 5 Applications of FIMB

In this section, we demonstrate how **FIMBs** can be adopted for the two applications, such as HTTPS firewall and encrypted DNS filtering. In particular, we define how the policy is set to extract the selective blocks (in §4.3) for inspection.

### 5.1 HTTPS Firewalling

We demonstrate how **FIMBs** can inspect outgoing encrypted connections containing HTTP/1.x requests, where 'x' indicates the version number, such as HTTP/1.0, HTTP/1.1, and HTTP/1.2. Our approach begins with analyzing the start of the HTTP/1.x request which verifies that the request employs HTTP/1.x. It is important to note that the current study of **FIMBs**, similar to **ZKMBs**, does not capture HTTP versions that operate using a frame-based protocol, such as HTTP/2 and HTTP/3, as these require different inspection techniques.

It is well-known that the first line of any HTTP request must end with the version, e.g. `HTTP/1.1`, and the lines are delimited with two-byte sequence `\r\n`. Suppose the block size is 128 bits, which equates to 16 bytes/characters (since 1 byte = 8 bits). That said, while we are inspecting the first 16-byte block, such as `GET /HTTP/1.1\r\n` (as shown in right-hand side of Fig. 2), which includes the first line of the HTTP request, up to and including the `\r\n` that signifies the end of the line.

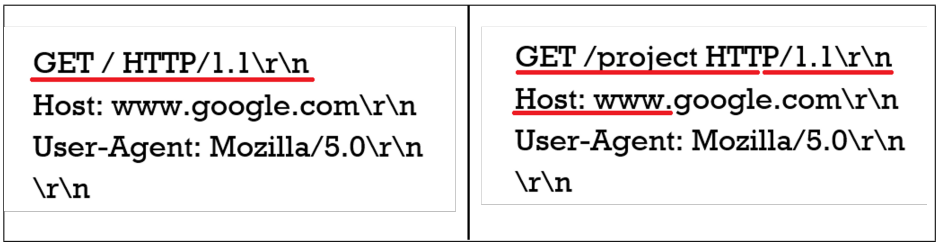


**Fig. 2.** Examples of HTTP/1.1 Requests. The left-hand side shows requests whose first line contains 16 bytes/characters. The right-hand side shows requests whose first line contains more than 16 bytes/characters.

For the case that `\r\n` is not in the first block (as shown in right hand-side of Fig. 2), nor exactly in the $j$-th, such that `GET /project HTT`, we can release the block next to it, i.e. $(j+1)$-th block that contains `P/1.1\r\nHost:www..` In this case, **MBs** observe the $j$-th and $(j+1)$-th block in plain, such that `GET /project HTTP/1.1\r\nHost:www.` are two blocks messages in 32 bytes.

### 5.2 Encrypted DNS

This section discusses the two types of encrypted DNS: DNS-over-TLS (DoT) and DNS-over-HTTPS (DoH). We then define the common policy required and explain how **FIMBs** operate to detect if **C** is accessing the blocklist. We assume that all DNS requests are encrypted using AEAD encryption under TLS 1.3.

In general, both types of inspections in **FIMBs** are quite similar. The process involves selecting the $j$-th block that contains the keyword associated with the DNS requests. We then release the keystream $k_j = E_K(IV_j)$ where $IV_j = IV||j+1$ to MB for decryption and inspection.

For DoT, the policy of **FIMBs** can enforce **C** to filter DNS requests. It is important to note that in ordinary DNS queries (without TLS), the DNS request is located at the 13th byte of the request. However, when DNS queries are sent over TLS, they are preceded by a two-byte length indicator [32]. As a result, the inspection should begin at byte 15th.

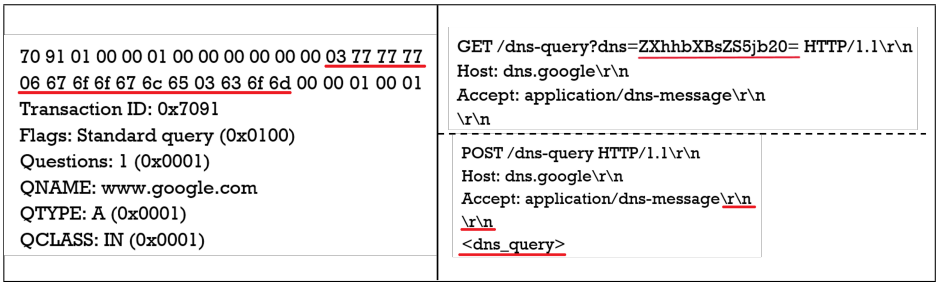


**Fig. 3.** Examples of DNS Requests. The left-hand side shows DoT requests. The right-hand side shows DoH requests using the GET and POST methods, respectively.

For DoH, the requests can be made using either HTTP GET or POST. In the scenario of HTTP GET, the process is straightforward, which is similar to HTTP firewalling. The policy enforces **C** to extract the first block by determining whether it is a GET request and then analyzing the $j$-th block(s) in the first line that starts after `?dns=`. Note that the DNS query in DoH is encoded in base64 format, we need to release the sequence of blocks until the delimiter `\r\n` is encountered.

For HTTP POST in DoH, a mixed approach is required, drawing from the previous examples. The policy enforces **C** to extract the first block to determine whether it is a POST request. **C** then looks for the delimiter `\r\n\r\n`, which marks the end of the HTTP headers. The subsequent filtering process depends on the DNS format, which is similar to what was discussed for DoT.

## 6 Performance Evaluation

Our experiment extensively leverages existing frameworks. For TLS protocol, we utilize tlslite-ng [1]. For AES-128-GCM, we employ the available source to extract the keystreams necessary for subsequent MPC operations (inspection and proof). For MPC, we rely on MP-SPDZ [34] using the semi-honest protocol. Most of the code is written in Python, with some source code from **ZKMBs** [28] utilized for application experiments, e.g. HTTPS firewalling and encrypted DNS filtering. The experiment for our **FIMBs** is conducted on a laptop running Ubuntu OS, equipped with an 8-core 3.8 GHz AMD Ryzen 7 7840HS CPU and 8GB of RAM. The source code is available at `https://github.com/FI-MB/FIMB`.

The policy comprises a set of algorithms, as detailed in (§5), designed to enforce the client's selection of blocks for HTTP firewalling or DNS filtering. Furthermore, the policy mandates that the client selects AES-128-GCM as the encryption algorithm during the TLS handshake, which uses a 128-bit key. This means that every message block has a size of 16 bytes. A one-time setup cost is required for the MPC, which takes approximately 3 seconds (s). This setup is performed upon joining the network and installing the extension.

The Establishment stage mirrors a standard TLS handshake, using either TLS 1.2 or TLS 1.3. The derived shared secret, which generates the session key and nonce, is subsequently utilized in the Selection and Extraction stage.

During the Selection and Extraction stage, the client has precomputed TLS ciphertext for certain well-formatted messages using the AES-128-GCM. In accordance with the policy, the client retrieves an index list for inspection. This index list is then employed to generate keystreams, enabling middleboxes (MBs) to decrypt the specified message blocks.

**Cost of Evaluation Phase.** The computational cost for generating proofs is linear to the message block(s). In particular, if we are proving for $k$-block(s) out of $n$-block(s) of the message $\boldsymbol{M} = (M_1, \cdots, M_n)$, the cost of proving using MPC in [34] is $k\times$ boolean check and $k + 2\times$ AES. The following table shows the efficiency of different blocks. Notably, a minimum of ≈300 milliseconds (ms) is required to prove a single block inspection. The cost increases ≈90 ms per additional block. We also compare this overhead to **ZKMBs** [28], using conservative estimates of their performance derived from the reported results for the same applications, such as HTTP firewalling and DNS filtering, in Figure 4, 5, and 6.

| Block(s) | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| ms | 296 | 391 | 488 | 586 | 676 | 765 | 867 |

For HTTP firewalling, if we inspect the HTTP version (typically involving 1–2 blocks), the proof generation cost is approximately 300–400 ms. We summa-

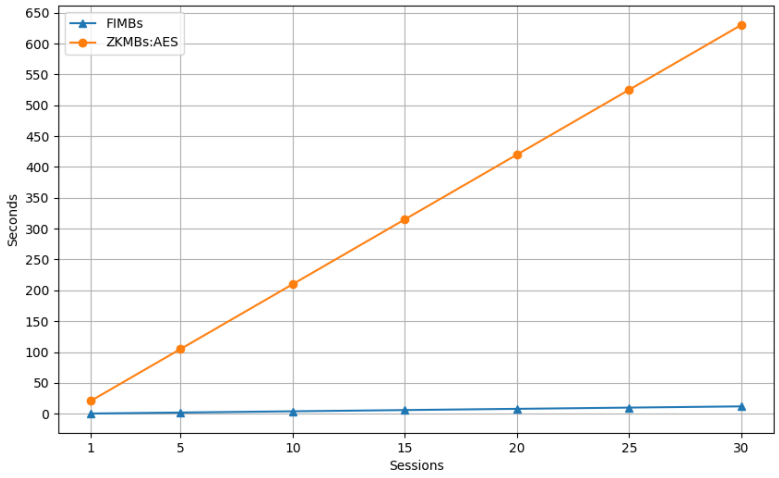


**Fig. 4.** This graph shows the additional computational overheads introduced when adopting either **FIMBs** and **ZKMBs** when a client accesses 30 distinct HTTPS websites, each requiring a new TLS handshake. Both protocols use AES-128-GCM. The overhead for **FIMBs** remains consistently low across all sessions, while **ZKMBs** show a sharp linear increase, surpassing 600 seconds by the 30th session, indicating significantly higher overhead as the number of sessions grows.

rize the performance evaluation for distinct session connections and resumption session connections in Figure 4 and Figure 5, respectively.

The size of a DNS request can vary. However, most DNS requests are relatively small, typically ranging from 6 to 86 bytes (from the blocklist [48]), which corresponds to 1–6 blocks. This size also depends on whether the request is base64-encoded.

In DoH (DNS over HTTPS), we additionally need to inspect the first block of the HTTP request (to determine whether it is a GET or POST request). This means that if $k$ is the possible block size of the DNS request, we need to inspect $k + 1$ blocks. Consequently, the proof generation time ranges from 400–900 ms if the DNS request is about 1-6 blocks.

In comparison to our **FIMBs**, the average generation and validation time is approximately ≈400 ms. Under a blocklist containing two million domains [48], **ZKMBs** require an initial setup time of two hours during the first connection. After this, session resumption (re-establishing a session key for multiple subsequent connections in TLS) takes 3 s for each proof generation. Although the verification time is only 2–5 ms, the trade-off in efficiency for clients and MBs is significant.

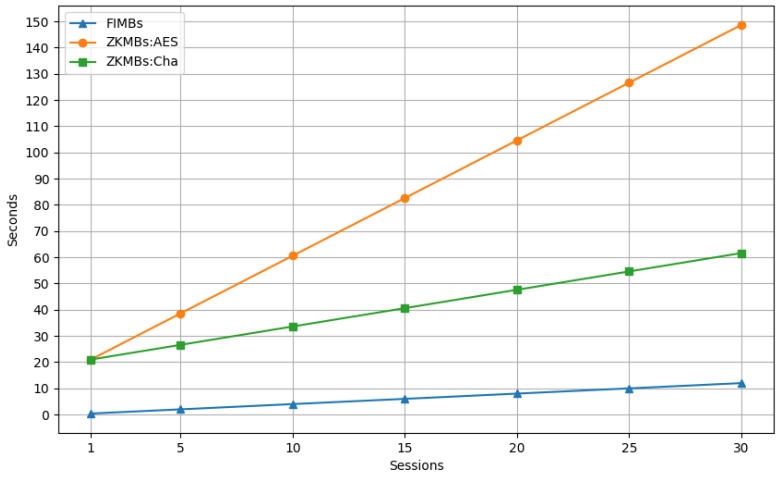


**Fig. 5.** This graph compares the additional computational overheads of **FIMBs** and **ZKMBs** (using either AES-128-GCM or ChaCha20/Poly1305) when a client accesses the same HTTPS website 30 times when session resumption is enabled. **FIMBs** maintain consistently low overhead across all sessions, while **ZKMBs** exhibit a gradual linear increase, with ChaCha20/Poly1305 performing better than AES-128-GCM but still showing higher overhead compared to **FIMBs**. It is important to note that the first session requires a TLS handshake, which includes the setup for **ZKMBs**, contributing to the initial overhead.

## 7 Security

The proposed **FIMBs** protocol is designed under a selective inspection setting (§3.1). Specifically, a semi-honest **MB** may observe only selective block(s) depending on the policy while keeping the remaining block(s) private. This said a dishonest client needs only to forge a few pieces of keystreams that pass the verification. While such forgery may appear straightforward, we will demonstrate its infeasibility in the subsequent analysis, owing to the robustness of the underlying cryptographic primitives.

First, we highlight that **FIMBs** do not make any modifications to the TLS protocol (§2.1). This means the shared secret established by client **C** and server **S** is identically based on TLS handshake. In our case, the derived session key and nonce $(K, IV)$ for the AES-128-GCM scheme (§2.2) remain as secure as the TLS [60, 17].

### 7.1 Security Against Dishonest Client

The proposed **FIMBs** protocol is secure against dishonest client **C** if the underlying MPC is secure under a semi-honest model [34]. Recall that **FIMBs** can

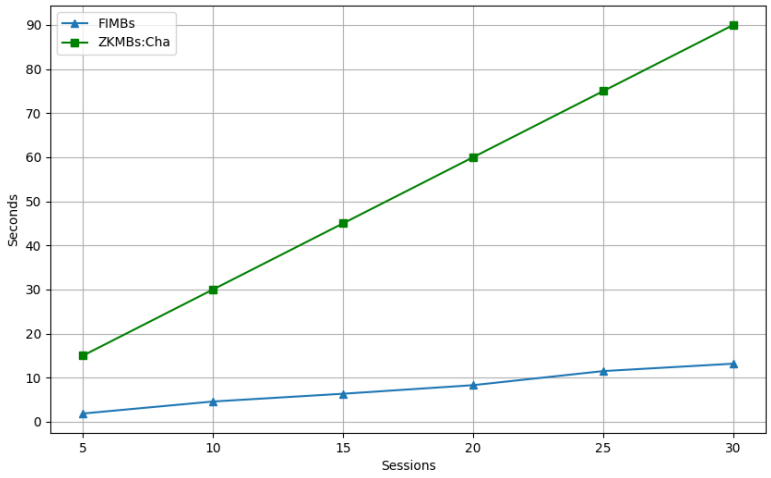


**Fig. 6.** This graph demonstrates the use case of DoT using **FIMBs** and **ZKMBs** (assuming the setup is already completed). It shows a client sending 30 different DNS queries to a DNS server after establishing an initial TLS session. The **ZKMBs** use ChaCha20/Poly1305, which generally has better performance as shown in Figure 5, they still result in higher computational overheads compared to **FIMBs** across all sessions.

detect if dishonest **C** that release some chosen keystreams $k'_j$ for some $j$-th block of ciphertext $C_j$ resulting in forged message $M'_j = C_j \oplus k'_j$ that obeys the policy, but the actual message $M_j$ violates the policy. We note that this is orthogonal to the *key commitment attacks* (§3.3) which **ZKMBs** protocol captures this by introducing the *channel opening sub-circuit* (§3.3). As mentioned, **FIMBs** protocol is not vulnerable to commitment attacks as **MB** inspects block(s) of messages in plain, so any random look messages shall be discarded.

There are possible ways to release chosen keystreams that result **MB** decrypts crafted message $M'$ that obeys the policy. They are **chosen keystreams attacks I & II** which has been discussed in (§3.3). The solution we adopt to resolve these attacks is by using any secure MPC to check the derivation of keystream with the TLS ciphertext tuple $CT = (AD, \boldsymbol{C}, T)$ based on generated proof tuple $\Pi = (IV, \boldsymbol{k} = (k_j, \cdots, k_{j_L}), L, H, k_0)$. This mechanism reflects the AEAD schemes with *encrypt-then-authenticate* construction (§2.2) that checks the integrity of $CT$, the algorithm to compute some block(s) of messages $M_j = C_j \oplus E_K(IV||j+1) = C_j \oplus k_j$ and authentication tag $T = GHASH(AD, \boldsymbol{C}, E_K(0)) \oplus E_K(IV||1) = GHASH(AD, \boldsymbol{C}, H) \oplus k_0$.

The security guarantee in this context is derived from the underlying MPC protocol, which executes the Boolean check and the AES block cipher. In particular, during the evaluation phase between **C** and **MB**, the input of session key

$X = K$ from **C** is bonded by the MPC program (**MB** can realize and abort if there are any changes to this step). By incorporating the Boolean check within the MPC protocol, we can ensure that both **C** and **MB** utilize the correct session nonce for the $j$-th block, where $IV_j = IV||j+1$, in the AES block cipher. This allows the resulting keystreams to be verified as $k_j = E_K(IV_j)$. This demonstrates that **C** adheres to the protocol, provided the underlying MPC protocol is secure.

### 7.2 Security Against Semi-Honest Middlebox

The proposed **FIMBs** protocol is secure against semi-honest middleboxes if the underlying AEAD scheme with *encrypt-then-authenticate* construction, i.e. AES-128-GCM [23], is indistinguishability under chosen-plaintext attacks (IND-CPA) secure. Revealing a single block of plaintext $M_j$ in AES-128-GCM does not compromise the session key $K$ and the remaining message blocks $\boldsymbol{M} = (M_1, \cdots, M_{j-1}, M_{j+1}, \cdots, M_n) \backslash \{M_j\}$. Each block is encrypted independently with a different nonce with counter mechanism $IV_j = IV||j+1$, i.e. $C_j = M_j \oplus E_K(IV_j)$. Additionally, the authentication feature of GCM ensures that tampering with the ciphertext is detectable $T = GHASH(AD, \boldsymbol{C}, E_K(0)) \oplus E_K(IV||1)$, preserving the overall confidentiality and integrity of the ciphertext.

Revealing a single block of plaintext does not compromise the session key $K$. In **FIMBs**, the client **C** does not release the session key $K$ in AES-GCM, which is used to generate keystream $k_j = E_K(IV_j)$ via the AES block cipher on a nonce with a session nonce and $j$-th block index $IV_j = IV||j+1$. Without the knowledge of $K$, even though an adversary obtains the session nonce $IV$ and some message and ciphertext pairs $(M_j, C_j)$ with $k_j = M_j \oplus C_j$, we see $K$ remains secret as it is due to the underlying AES block cipher $k_j = E_K(IV_j)$. Notably, $K$ remains secure if the input nonces are unique and distinct, which is how AES-128-GCM works, the input session nonce $IV$ is different for every block.

Knowing $j$-th block of message and ciphertext pair $(M_j, C_j)$ in AES-128-GCM does not directly help an adversary decrypt other ciphertext block(s). Again, as AES-128-GCM uses a unique nonce combined with a counter for each block $IV_j = IV||j+1$, ensuring that each block is encrypted with a different input. Therefore, knowing keystream $k_j$ for $j$-th block cannot help in deducing for another block. In short, each block in GCM mode is encrypted independently, meaning the encryption of one block does not reveal information about the encryption of other blocks. While the AES-128-GCM is an AEAD scheme with *encrypt-then-authenticate* construction. If the integrity check fails (i.e. if the ciphertext or additional authenticated data has been tampered with), the decryption will not proceed successfully, and the ciphertext will be rejected.

## 8 Related Work

**Searchable Encryption.** Sherry *et al.* [63] introduced BlindBox, an early framework for privacy-preserving deep packet inspection designed to directly

| Approaches | Inspection by Decryption | Server Collaboration | TLS Modification | MB Trust Assumption | DNS Filtering | |
|---|---|---|---|---|---|---|
| | | | | | Client Overhead | MB Overhead |
| MitM | Yes | No | No | Trusted | Low | Low |
| Access Control | Yes | Yes | Yes | - | Low | Low |
| Trusted Execution Environment | Yes | No | No | Trusted | Low | Low |
| Machine Learning | No | No | No | Semi-Honest | - | - |
| Searchable Encryption | No | Yes | No | Semi-Honest | Medium + High-Setup | Medium |
| **ZKMBs** | No | No | No | Semi-Honest | High + High-Setup | Medium |
| **FIMBs** | Selective-Blocks | No | No | Semi-Honest | Medium + One-Time-Setup | High |

**Table 1.** A summary table of the existing approaches. **FIMBs** offer a distinctive middle ground by enabling selective-blocks decryption. Unlike methods that necessitate significant modifications to existing TLS protocols or rely on trusted hardware, **FIMBs** maintain compatibility with the standard TLS protocol, enhancing adaptability across diverse network environments. Similar to **ZKMBs**, **FIMBs** do not require any modifications on the server side, further simplifying their deployment and ensuring seamless integration with existing infrastructures without the need for extensive changes. In the DNS filtering application, low-overhead approaches necessitate decryption, rendering machine learning inapplicable; client overhead represents the generation of encrypted traffic, while MB overhead reflects the middlebox's inspection processes, with trade-offs involving setup time variations, and searchable encryption, ZKMBs, and FIMBs effectively balancing efficiency and setup complexity.

inspect encrypted traffic. However, the reliance on garbled circuits and oblivious transfer for each session renders BlindBox impractical. Building on BlindBox as a foundational component, Lan *et al.* [41] proposed Embark, which employs a trusted enterprise gateway for privacy-preserving detection within a cloud-based middlebox (MB) environment. In Embark, the enterprise gateway must be fully trusted and have access to both the traffic content and detection rules, while the client is not required to perform any operations.

SPABox by Fan *et al.* [24] uses a public key approach with an oblivious pseudo-random function and a modified garbled circuit for rule matching. Yuan *et al.* [76] proposed a scheme that is more efficient than BlindBox but requires the server to first register with the administration service of the enterprise hosting the client. For regular expression matching, the scheme deploys a variant of the garbled circuit.

These approaches allow MBs to perform policy checks on encrypted data, focusing on keyword searches. However, they all require modifications to servers and rely on server checks to ensure consistency between the TLS plaintext and the encryptions via other channels. Without this, clients could send policy-violating traffic while appearing compliant. Several subsequent works [55, 54, 37, 43, 39, 45, 75, 40, 46] have been proposed under this setting.

**Access Control.** There are also proposals that employ a client-server accountable model, ensuring that both the client and server can recognize and authenticate all MBs positioned between them. Naylor *et al.* [52] introduced a framework for this model, known as mcTLS. This framework adapts the existing TLS protocol to enable the client, the MBs, and the server to establish authenticated and secure channels, facilitating the exchange of read and write secret keys along with the session key. However, adopting mcTLS necessitates replacing the existing TLS protocol entirely.

To address this issue, Lee *et al.* [42] proposed the maTLS framework, which maintains compatibility with the TLS protocol. Instead of altering the underlying protocol, maTLS introduces extensions that work seamlessly with TLS. Other related work includes [71, 44, 7, 10, 2]. While this approach allows for more granular trade-offs compared to completely disabling encryption, it still significantly compromises user privacy and requires changes on the server side.

**Machine Learning.** Other related work includes proposals for analyzing encrypted traffic using machine learning (ML) techniques [72] that do not directly inspect the encrypted payloads. Anderson *et al.* [4–6] proposed methods for malware detection that utilize various header information and metadata. Ede *et al.* [67] generated mobile application fingerprints from encrypted network traffic network traffic. However, there are inherent limitations to what can be analyzed using ML techniques [15], and these methods necessitate the continuous input of high-quality labelled data for training.

**Trusted Execution Environment.** Trusted hardware has also been employed for privacy-preserving deep packet inspection, with many proposals leveraging the secure enclave of Intel SGX. The primary concept is to provide the trusted hardware within the MBs with the session key, enabling decryption, inspection, and re-encryption within the trusted hardware. Han *et al.* [30] proposed a scheme called SGX-Box based on this approach. Additionally, there are proposals for secure Network Function Virtualization (NFV) systems that use trusted hardware for deep packet inspection, such as SafeBricks by Poddar *et al.* [58], ShieldBox by Trach *et al.* [65], and LightBox by Duan *et al.* [21]. Moreover, Naylor *et al.* proposed mbTLS [51], which enhances mcTLS (access control) by employing trusted hardware to implement MB functionality between each hop. Despite their secure architecture, trusted hardware has been found to have a significant attack surface, making them vulnerable to various types of attacks [27, 50, 61, 68, 66].

## 9 Summary

The proposed **FIMBs** represent a significant advancement in achieving a balance between efficiency and privacy for middleboxes (MBs) operating over TLS traffic that use authenticated encryption with associated data (AEAD) schemes (*encrypt-then-authenticate* construction). By selectively revealing only necessary keystreams for specific data blocks, **FIMBs** enable the inspection of encrypted traffic while safeguarding the privacy of the remaining content. Although the

current **FIMBs** only target certain applications such as HTTP firewalling and DNS filtering (§1.1), this selective inspection approach effectively addresses the inherent privacy concerns of the traditional **MitM** approach, which involves decrypting all traffic and, thereby, compromising user privacy.

Our experimental results, instantiated using AES-128-GCM, show that **FIMBs** offer significant improvements in efficiency compared to **ZKMBs**. While **ZKMBs** provide comprehensive privacy guarantees for content, they are hindered by impractical overheads during session setup and proof generation. For example, in the HTTP version firewalling, **FIMBs** introduce only a 300 ms overhead for validating keystreams for a single block, which is substantially lower than the 21 seconds required by **ZKMBs** for setup and proof generation (excluding validation, which takes only 2–5 ms). From the client's perspective, this level of overhead is impractical, making **FIMBs** a more viable solution.

A key innovation of **FIMBs** is its ability to circumvent the costly session setup associated with **ZKMBs** by employing techniques inspired by AEAD schemes with *encrypt-then-authenticate* construction. Enabling MBs to inspect specific blocks of encrypted traffic without requiring the decryption of the entire session, significantly reduces the trade-off between efficiency and privacy.

However, while **FIMBs** offer enhanced efficiency, they are not without limitations. The process of validating keystreams, particularly when utilizing MPC, can be resource-intensive. Although MPC enhances security by ensuring the correct generation of keystreams, the computational demands of this process may present challenges in large-scale deployments or in environments with limited resources. Table 1 summarizes our observation.

## 10 Conclusion

In conclusion, **FIMBs** represent a practical solution for facilitating secure and efficient inspection of TLS-encrypted traffic. By achieving a balance between privacy and performance, it offers a viable alternative to the resource-intensive **ZKMBs** and the privacy-invasive **MitM** approaches. Future research could further optimise the computational overhead associated with keystream validation and expand the applicability of **FIMBs** to a wider array of network security scenarios.

## References

1. tlslite-ng:tls implementation in pure python (2021), `https://github.com/tlsfuzzer/tlslite-ng`
2. Ahn, T., Kwak, J., Kim, S.: mdtls: How to make middlebox-aware tls more efficient? In: International Conference on Information Security and Cryptology. pp. 39–59. Springer (2023)
3. Akdemir, K., Dixon, M., Feghali, W., Fay, P., Gopal, V., Guilford, J., Ozturk, E., Wolrich, G., Zohar, R.: Breakthrough aes performance with intel aes new instructions. White paper, June **12**, 217 (2010)

4. Anderson, B., McGrew, D.A.: Identifying Encrypted Malware Traffic with Contextual Flow Data. In: Freeman, D.M., Mitrokotsa, A., Sinha, A. (eds.) Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, AISec@CCS 2016, Vienna, Austria, October 28, 2016. pp. 35–46. ACM (2016). https://doi.org/10.1145/2996758.2996768, http://doi.acm.org/10.1145/2996758.2996768
5. Anderson, B., McGrew, D.A.: Machine Learning for Encrypted Malware Traffic Classification: Accounting for Noisy Labels and Non-Stationarity. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017. pp. 1723–1732. ACM (2017). https://doi.org/10.1145/3097983.3098163, http://doi.acm.org/10.1145/3097983.3098163
6. Anderson, B., Paul, S., McGrew, D.A.: Deciphering malware's use of TLS (without decryption). J. Computer Virology and Hacking Techniques **14**(3), 195–211 (2018). https://doi.org/10.1007/s11416-017-0306-6, https://doi.org/10.1007/s11416-017-0306-6
7. Baek, J., Kim, J., Susilo, W.: Inspecting tls anytime anywhere: a new approach to tls interception. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security. pp. 116–126 (2020)
8. Barker, E., Souppaya, M., Smid, M.: Guidelines for the selection, configuration, and use of transport layer security (TLS) implementations. NIST Special Publication 800-52 Revision 2, National Institute of Standards and Technology (2019). https://doi.org/10.6028/NIST.SP.800-52r2, https://doi.org/10.6028/NIST.SP.800-52r2
9. Bellare, M., Hoang, V.T., Keelveedhi, S., Rogaway, P.: Efficient garbling from a fixed-key blockcipher. In: 2013 IEEE Symposium on Security and Privacy. pp. 478–492. IEEE (2013)
10. Bhargavan, K., Boureanu, I., Delignat-Lavaud, A., Fouque, P.A., Onete, C.: A Formal Treatment of Accountable Proxying over TLS. In: 2018 IEEE Symposium on Security and Privacy, SP 2018, San Francisco, CA, USA, May 21-23, 2018. pp. 339–356. IEEE Computer Society (2018)
11. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 337–367. Springer (2015)
12. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: Improvements and extensions. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 1292–1303 (2016)
13. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. ACM Transactions on Computation Theory (TOCT) **6**(3), 1–36 (2014)
14. de Carné de Carnavalet, X., Mannan, M.: Killed by Proxy: Analyzing Client-end TLS Interception Software. In: 23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016. The Internet Society (2016), http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2017/09/killed-proxy-analyzing-client-end-tls-interception-software.pdf
15. de Carné de Carnavalet, X., van Oorschot, P.C.: A survey and analysis of tls interception mechanisms and motivations: Exploring how end-to-end tls is made "end-to-me" for web traffic. ACM Computing Surveys **55**(13s), 1–40 (2023)
16. Chan, J., Rogaway, P.: On committing authenticated-encryption. In: European Symposium on Research in Computer Security. pp. 275–294. Springer (2022)

17. Dierks, T., Rescorla, E.: The transport layer security (tls) protocol version 1.2. RFC 5246 (August 2008), https://www.rfc-editor.org/rfc/rfc5246.txt
18. Dixon, C., Uppal, H., Brajkovic, V., Brandon, D., Anderson, T., Krishnamurthy, A.: {ETTM}: A scalable fault tolerant network manager. In: 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11) (2011)
19. Dodis, Y., Grubbs, P., Ristenpart, T., Woodage, J.: Fast message franking: From invisible salamanders to encryptment. In: Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38. pp. 155–186. Springer (2018)
20. Duan, H., Wang, C., Yuan, X., Zhou, Y., Wang, Q., Ren, K.: Lightbox: Full-stack protected stateful middlebox at lightning speed. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. CCS '19, ACM (Nov 2019). https://doi.org/10.1145/3319535.3339814, http://dx.doi.org/10.1145/3319535.3339814
21. Duan, H., Yuan, X., Wang, C.: LightBox: SGX-assisted Secure Network Functions at Near-native Speed. CoRR **abs/1706.06261** (2017), http://arxiv.org/abs/1706.06261
22. Durumeric, Z., Ma, Z., Springall, D., Barnes, R., Sullivan, N., Bursztein, E., Bailey, M., Halderman, J.A., Paxson, V.: The Security Impact of HTTPS Interception. In: 24th Annual Network and Distributed System Security Symposium, NDSS 2017, San Diego, California, USA, February 26 - March 1, 2017. The Internet Society (2017), https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/security-impact-https-interception/
23. Dworkin, M.: Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology (2007), https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
24. Fan, J., Guan, C., Ren, K., Cui, Y., Qiao, C.: SPABox: Safeguarding Privacy During Deep Packet Inspection at a MiddleBox. IEEE/ACM Trans. Netw. **25**(6), 3753–3766 (2017). https://doi.org/10.1109/TNET.2017.2753044, https://doi.org/10.1109/TNET.2017.2753044
25. Goltzsche, D., R¨usch, S., Nieke, M., Vaucher, S., Weichbrodt, N., Schiavoni, V., Aublin, P.L., Cosa, P., Fetzer, C., Felber, P., et al.: Endbox: Scalable middlebox functions using client-side trusted execution. In: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 386–397. IEEE (2018)
26. Gong, D., Tran, M., Shinde, S., Jin, H., Sekar, V., Saxena, P., Kang, M.S.: Practical verifiable in-network filtering for ddos defense. In: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS). pp. 1161–1174. IEEE (2019)
27. Götzfried, J., Eckert, M., Schinzel, S., Müller, T.: Cache attacks on intel sgx. In: Proceedings of the 10th European Workshop on Systems Security. pp. 1–6 (2017)
28. Grubbs, P., Arun, A., Zhang, Y., Bonneau, J., Walfish, M.: {Zero-Knowledge} middleboxes. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 4255–4272 (2022)
29. Grubbs, P., Lu, J., Ristenpart, T.: Message franking via committing authenticated encryption. In: Advances in Cryptology–CRYPTO 2017: 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20–24, 2017, Proceedings, Part III 37. pp. 66–97. Springer (2017)

30. Han, J., Kim, S.M., Ha, J., Han, D.: SGX-Box: Enabling Visibility on Encrypted Traffic using a Secure Middlebox Module. In: Chen, K., Padhye, J. (eds.) Proceedings of the First Asia-Pacific Workshop on Networking, APNet 2017, Hong Kong, China, August 3-4, 2017. pp. 99–105. ACM (2017). https://doi.org/10.1145/3106989.3106994, http://doi.acm.org/10.1145/3106989.3106994
31. Han, J., Kim, S., Cho, D., Choi, B., Ha, J., Han, D.: A secure middlebox framework for enabling visibility over multiple encryption protocols. IEEE/ACM Transactions on Networking **28**(6), 2727–2740 (2020)
32. Hu, Z., Zhu, L., Heidemann, J., Mankin, A., Wessels, D., Hoffman, P.: Specification for dns over transport layer security (tls). Request for Comments (RFC) RFC 7858, Internet Engineering Task Force (IETF) (May 2016), https://www.rfc-editor.org/rfc/rfc7858
33. Jarmoc, J., Unit, D.: Ssl/tls interception proxies and transitive trust. Black Hat Europe (2012)
34. Keller, M.: MP-SPDZ: A versatile framework for multi-party computation. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (2020). https://doi.org/10.1145/3372297.3417872, https://doi.org/10.1145/3372297.3417872
35. Keller, M., Orsini, E., Scholl, P.: Actively secure ot extension with optimal overhead. In: Annual Cryptology Conference. pp. 724–741. Springer (2015)
36. Keller, M., Orsini, E., Scholl, P.: Mascot: faster malicious arithmetic secure computation with oblivious transfer. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 830–842 (2016)
37. Kim, J., Camtepe, S., Baek, J., Susilo, W., Pieprzyk, J., Nepal, S.: P2dpi: practical and privacy-preserving deep packet inspection. In: Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security. pp. 135–146 (2021)
38. Kuvaiskii, D., Chakrabarti, S., Vij, M.: Snort intrusion detection system with intel software guard extension (intel sgx). arXiv preprint arXiv:1802.00508 (2018)
39. Lai, S., Yuan, X., Liu, J., Yi, X., Li, Q., Liu, D., Nepal, S.: Oblivsketch: Oblivious network measurement as a cloud service. In: Usenix Network and Distributed System Security Symposium 2021. Internet Society (2021)
40. Lai, S., Yuan, X., Sun, S.F., Liu, J.K., Steinfeld, R., Sakzad, A., Liu, D.: Practical encrypted network traffic pattern matching for secure middleboxes. IEEE Transactions on Dependable and Secure Computing **19**(4), 2609–2621 (2021)
41. Lan, C., Sherry, J., Popa, R.A., Ratnasamy, S., Liu, Z.: Embark: Securely Outsourcing Middleboxes to the Cloud. In: Argyraki, K.J., Isaacs, R. (eds.) 13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016. pp. 255–273. USENIX Association (2016), https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/lan
42. Lee, H., Smith, Z., Lim, J., Choi, G., Chun, S., Chung, T., Kwon, T.T.: matls: How to make tls middlebox-aware? In: NDSS (2019)
43. Li, H., Ren, H., Liu, D., Shen, X.S.: Privacy-enhanced deep packet inspection at outsourced middlebox. In: 2018 10th International Conference on Wireless Communications and Signal Processing (WCSP). pp. 1–6. IEEE (2018)
44. Li, J., Chen, R., Su, J., Huang, X., Wang, X.: Me-tls: middlebox-enhanced tls for internet-of-things devices. IEEE Internet of Things Journal **7**(2), 1216–1229 (2019)
45. Liu, C., Cui, Y., Tan, K., Fan, Q., Ren, K., Wu, J.: Building generic scalable middlebox services over encrypted protocols. In: IEEE INFOCOM 2018-IEEE conference on computer communications. pp. 2195–2203. IEEE (2018)

46. Liu, Q., Peng, Y., Jiang, H., Wu, J., Wang, T., Peng, T., Wang, G.: Slimbox: Lightweight packet inspection over encrypted traffic. IEEE Transactions on Dependable and Secure Computing **20**(5), 4359–4371 (2022)
47. Marquis-Boire, M., Dalek, J., McKune, S., Carrieri, M., Crete-Nishihata, M., Deibert, R., Omar Khan, S., Scott-Railton, J., Wiseman, G.: Planet blue coat: Mapping global censorship and surveillance tools (2013)
48. Mayfield, C.: my-pihole-blocklists/pi_blocklist_porn_all.list. `https://github.com/chadmayfield/my-pihole-blocklists` (2021), accessed: 2021
49. Menda, S., Len, J., Grubbs, P., Ristenpart, T.: Context discovery and commitment attacks: How to break ccm, eax, siv, and more. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 379–407. Springer (2023)
50. Murdock, K., Oswald, D., Garcia, F.D., Van Bulck, J., Gruss, D., Piessens, F.: Plundervolt: Software-based fault injection attacks against intel sgx. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 1466–1482. IEEE (2020)
51. Naylor, D., Li, R., Gkantsidis, C., Karagiannis, T., Steenkiste, P.: And Then There Were More: Secure Communication for More Than Two Parties. In: Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2017, Incheon, Republic of Korea, December 12 - 15, 2017. pp. 88–100. ACM (2017). https://doi.org/10.1145/3143361.3143383, `http://doi.acm.org/10.1145/3143361.3143383`
52. Naylor, D., Schomp, K., Varvello, M., Leontiadis, I., Blackburn, J., López, D.R., Papagiannaki, K., Rodríguez, P.R., Steenkiste, P.: Multi-Context TLS (mcTLS): Enabling Secure In-Network Functionality in TLS. In: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM 2015, London, United Kingdom, August 17-21, 2015. pp. 199–212. ACM (2015). https://doi.org/10.1145/2785956.2787482
53. Nilsson, A., Bideh, P.N., Brorsson, J.: A survey of published attacks on intel sgx. arXiv preprint arXiv:2006.13598 (2020)
54. Ning, J., Huang, X., Poh, G.S., Xu, S., Loh, J.C., Weng, J., Deng, R.H.: Pine: Enabling privacy-preserving deep packet inspection on tls with rule-hiding and fast connection establishment. In: Computer Security–ESORICS 2020, Guildford, UK. pp. 3–22. Springer (2020)
55. Ning, J., Poh, G.S., Loh, J.C., Chia, J., Chang, E.C.: Privdpi: Privacy-preserving encrypted traffic inspection with reusable obfuscated rules. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 1657–1670 (2019)
56. Nir, Y., Langley, A.: Chacha20 and poly1305 for ietf protocols. RFC 7539 (May 2015), `https://www.rfc-editor.org/rfc/rfc7539.txt`
57. O'Neill, M., Ruoti, S., Seamons, K., Zappala, D.: Tls proxies: Friend or foe? In: Proceedings of the 2016 Internet Measurement Conference. pp. 551–557 (2016)
58. Poddar, R., Lan, C., Popa, R.A., Ratnasamy, S.: {SafeBricks}: shielding network functions in the cloud. In: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). pp. 201–216 (2018)
59. Qualys, S.: Labs-ssl pulse. `https://www.ssllabs.com/ssl-pulse/` (2024), accessed: 2024-05-01
60. Rescorla, E.: The transport layer security (tls) protocol version 1.3. RFC 8446 (August 2018), `https://www.rfc-editor.org/rfc/rfc8446.txt`
61. Schwarz, M., Weiser, S., Gruss, D., Maurice, C., Mangard, S.: Malware guard extension: Using sgx to conceal cache attacks. In: Detection of Intrusions and

Malware, and Vulnerability Assessment: 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings 14. pp. 3–24. Springer (2017)
62. Security, C.: Egress filtering 101: What it is and how to do it (2015), https://www.calyptix.com/educational-resources/egress-filtering-101-what-it-is-and-how-to-do-it/
63. Sherry, J., Lan, C., Popa, R.A., Ratnasamy, S.: Blindbox: Deep packet inspection over encrypted traffic. In: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM 2015, London, United Kingdom, August 17-21, 2015. pp. 213–226 (2015)
64. Soghoian, C., Stamm, S.: Certified lies: Detecting and defeating government interception attacks against ssl (short paper). In: International Conference on Financial Cryptography and Data Security. pp. 250–259. Springer (2011)
65. Trach, B., Krohmer, A., Gregor, F., Arnautov, S., Bhatotia, P., Fetzer, C.: Shieldbox: Secure middleboxes using shielded execution. In: Proceedings of the Symposium on SDN Research. pp. 1–14 (2018)
66. Van Bulck, J., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R.: Foreshadow: Extracting the keys to the intel {SGX} kingdom with transient {Out-of-Order} execution. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 991–1008 (2018)
67. Van Ede, T., Bortolameotti, R., Continella, A., Ren, J., Dubois, D.J., Lindorfer, M., Choffnes, D., Van Steen, M., Peter, A.: Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In: Network and distributed system security symposium (NDSS). vol. 27 (2020)
68. Van Schaik, S., Minkin, M., Kwong, A., Genkin, D., Yarom, Y.: Cacheout: Leaking data on intel cpus via cache evictions. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 339–354. IEEE (2021)
69. Waked, L., Mannan, M., Youssef, A.: To intercept or not to intercept: Analyzing tls interception in network appliances. In: Proceedings of the 2018 on Asia Conference on Computer and Communications Security. pp. 399–412 (2018)
70. Wang, J., Hao, S., Li, Y., Hong, Z., Yan, F., Zhao, B., Ma, J., Zhang, H.: Tvids: Trusted virtual ids with sgx. China Communications **16**(10), 133–150 (2019)
71. Wilkens, F., Haas, S., Amann, J., Fischer, M.: Passive, transparent, and selective tls decryption for network security monitoring. In: IFIP International Conference on ICT Systems Security and Privacy Protection. pp. 87–105. Springer (2022)
72. Yamada, A., Miyake, Y., Takemori, K., Studer, A., Perrig, A.: Intrusion detection for encrypted web accesses. In: 21st International Conference on Advanced Information Networking and Applications Workshops (AINAW'07). vol. 1, pp. 569–576. IEEE (2007)
73. Yao, A.C.: Protocols for secure computations. In: 23rd annual symposium on foundations of computer science (sfcs 1982). pp. 160–164. IEEE (1982)
74. Yao, A.C.C.: How to generate and exchange secrets. In: 27th annual symposium on foundations of computer science (Sfcs 1986). pp. 162–167. IEEE (1986)
75. Yuan, X., Duan, H., Wang, C.: Assuring string pattern matching in outsourced middleboxes. IEEE/ACM Transactions on Networking **26**(3), 1362–1375 (2018)
76. Yuan, X., Wang, X., Lin, J., Wang, C.: Privacy-preserving deep packet inspection in outsourced middleboxes. In: 35th Annual IEEE International Conference on Computer Communications, INFOCOM 2016, San Francisco, CA, USA, April 10-14, 2016. pp. 1–9. IEEE (2016)
77. Zhang, C., DeStefano, Z., Arun, A., Bonneau, J., Grubbs, P., Walfish, M.: Zombie: Middleboxes that don't snoop. In: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24). pp. 1917–1936 (2024)